

Project Team Members

Sr. No	Name	Registration No
1	Muhammad Madni	B23F0309SE065
2	Muhammad Wasif	B23F0396SE006
3	Muhammad Zohaib	B23F0515SE005
4	Duad Latif	B23F0195SE011
5	Haneeza Liaqat Awan	B23F0493SE010

Code Quality Checker

1. Introduction

1.1 Project Overview

Code Quality Checker is a full-stack web application that analyzes Java source code using Artificial Intelligence and provides detailed feedback about code quality. The system allows users to paste Java code into a web interface and receive intelligent suggestions regarding errors, code structure, readability, and best coding practices.

The project uses Spring Boot for backend development and Spring AI for communication with AI models. The frontend is built using HTML, CSS, and JavaScript to provide a simple and interactive user experience.

The application acts like an automated code reviewer that helps developers improve their coding standards and understand programming mistakes more effectively.

2. Problem Statement

Manual code review takes time and requires experienced developers. Beginners often struggle to identify mistakes in their code and understand best programming practices.

This project solves this problem by creating an AI-powered system that automatically reviews Java code and provides understandable feedback and improvement suggestions.

3. Objectives

The main objectives of this project are:

- To automate the code review process
- To help beginners understand coding mistakes
- To improve code readability and maintainability
- To integrate Artificial Intelligence into software engineering
- To build a real-world full-stack application using Spring Boot and Spring AI

4. System Features

4.1 Code Input

Users can:

- Paste Java code into a textarea
- Submit code for analysis

4.2 AI-Based Code Analysis

The system analyzes code using AI based on:

- Syntax issues
- Logical mistakes
- Code smells
- Naming conventions
- Readability
- Code structure

4.3 Issue Detection

The system identifies issues such as:

- Long methods
- Poor variable naming
- Unused code
- Redundant statements
- Lack of proper formatting

4.4 Suggestions and Recommendations

For every issue detected, the system provides:

- Explanation of the problem
- Suggested improvements
- Best practice recommendations

4.5 Output Display

The frontend displays:

- Full analysis report
- Error explanations
- Improvement suggestions

5. System Architecture

The project follows a client-server architecture.

Frontend (HTML/CSS/JavaScript)



Spring Boot REST API



Service Layer



Spring AI Integration



AI Model (LLM)



Response Returned to Frontend

6. Modules of the System

6.1 Frontend Module

Responsibilities:

- User interface design
- Code input area
- Displaying analysis results

Technologies Used:

- HTML
- CSS
- JavaScript

6.2 Controller Module

Responsibilities:

- Handles HTTP requests
- Receives code from frontend

- Sends requests to service layer

Example Endpoint:

POST /api/check

6.3 Service Module

Responsibilities:

- Contains business logic
- Sends code to AI model
- Processes AI responses

This module communicates with Spring AI to generate intelligent code analysis.

6.4 Model Module

Responsibilities:

- Defines request and response structures
- Transfers data between layers

Example:

- CodeRequest
- CodeResponse

7. Technologies Used

Technology	Purpose
Java	Backend programming
Spring Boot	Backend framework
Spring AI	AI integration
HTML	Frontend structure
CSS	Frontend styling
JavaScript	Frontend functionality
REST API	Communication between frontend and backend

8. Workflow of the System

Step 1: User Inputs Code

The user pastes Java code into the web interface.

Step 2: Request Sent to Backend

JavaScript sends the code to the Spring Boot API using HTTP POST request.

Step 3: AI Processing

The backend sends the code to the AI model through Spring AI.

Step 4: Code Analysis

The AI analyzes the code and generates feedback.

Step 5: Response Returned

The backend sends the analysis result back to the frontend.

Step 6: Display Results

The frontend displays the detailed analysis report.

9. API Details

Endpoint

POST /api/check

Request Body

```
{
  "code": "public class Test {}"
}
```

Response

```
{
  "analysis": "The code structure is good but variable naming should be improved."
}
```

10. User Interface Description

The frontend interface contains:

- A textarea for code input

- A submit button
- A result display section

The design is simple and user-friendly to help beginners use the system easily.

11. Advantages of the System

- Fast automated code review
- Beginner-friendly explanations
- Improves coding practices
- Saves development time
- Easy to use interface
- Scalable architecture

12. Limitations

- Currently supports only Java code
- Depends on AI response quality
- Requires internet/API access for AI communication

13. Future Enhancements

Future improvements may include:

- Support for multiple programming languages
- Code quality scoring system
- User authentication system
- History tracking
- Integration with SonarQube
- Downloadable PDF reports
- Dark mode interface
- Real-time code suggestions

14. Conclusion

The Code Quality Checker project demonstrates how Artificial Intelligence can improve software development and code review processes. By integrating Spring Boot with Spring AI, the system provides intelligent code analysis and meaningful suggestions to developers.

This project is a practical implementation of modern full-stack development combined with AI technologies. It helps programmers write cleaner, more maintainable, and professional code while also serving as a learning tool for beginners.